

Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система и загрузчик

Цель работы

Изучить назначение дерева устройств, OpenSBI, U-Boot, конфигурации Extlinux и корневой файловой системы, а затем подготовить эти компоненты для последующей записи образа на носитель.

Оборудование и предварительные требования

- рабочий компьютер с ALT Linux и доступом в интернет;
- установленный кросс-тулчейн RISC-V;
- утилиты `git`, `make`, `patch`, `swig`, `dtc`, `file`, `xz`, `cpio`, `rsync`, `bison`, `flex` и зависимости для сборки U-Boot и OpenSBI;
- ядро, `.config` и DTB, полученные в лабораторной №4;
- не менее 10 Гбайт свободного места для исходного кода и загруженных компонентов.

Опасность повреждения

В этой работе не записывайте данные на microSD, не запускайте `mkfs` и не изменяйте таблицу разделов. Не заменяйте `/etc/apt/sources.list` ради получения одного пакета. Подготовка носителя выполняется в лабораторной №6.

Ключевые понятия

Дерево устройств

Определение

Device Tree (DT) — иерархическая структура данных, которая описывает оборудование платформы для ядра операционной системы.

Device Tree позволяет одному ядру работать на разных платах без перекompиляции при наличии подходящего описания оборудования. Дополнительные сведения приведены в статье на altlinux.org.

Определение

DTS (Device Tree Source) — человекочитаемый исходный файл описания дерева устройств с расширением `.dts` или включаемый фрагмент с расширением `.dtsi`.

Определение

DTB (Device Tree Blob) — бинарное представление дерева устройств, полученное компиляцией DTS и передаваемое ядру при загрузке.

Основным источником DTS служат исходный код Linux и репозитории разработчиков оборудования. В дереве ядра описания обычно находятся в каталоге `arch/архитектура/boot/dts/производитель/`.

Для Lichee RV Dock исходный файл расположен по пути:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dts
```

После сборки ядра соответствующий DTB находится здесь:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Практический ориентир. Если готового DTS для платы нет, отправной точкой может служить описание наиболее похожего устройства. Его адаптируют по документации SoC, электрической схеме и распиновке платы.

Цепочка загрузки

Определение

Загрузчик (bootloader) — программа, которая после включения системы подготавливает оборудование и загружает следующий программный компонент или ядро операционной системы.

Для одноплатных компьютеров широко применяется Das U-Boot, обычно называемый U-Boot.

Определение

SPL (Secondary Program Loader) — компактная первая стадия U-Boot, которая выполняет раннюю инициализацию, в том числе настраивает оперативную память для загрузки следующих компонентов.

Определение

OpenSBI — реализация интерфейса Supervisor Binary Interface для RISC-V, которая работает в привилегированном M-mode и предоставляет среде S-mode стандартные низкоуровневые сервисы.

Для Lichee RV Dock цепочку загрузки можно представить так:

1. После подачи питания встроенный BootROM процессора ищет загрузочный код на поддерживаемом носителе и считывает SPL из предусмотренной для него области.

2. SPL выполняет раннюю инициализацию, включая настройку DRAM, и загружает последующие компоненты комбинированного образа U-Boot.
3. OpenSBI запускается в M-mode, остаётся резидентным и передаёт управление U-Boot proper в S-mode.
4. U-Boot proper инициализирует необходимую периферию, читает конфигурацию загрузки и загружает ядро Linux и DTB.
5. U-Boot передаёт ядру управление, а ядро обращается к сервисам OpenSBI через вызовы SBI, например при перезагрузке системы.

BootROM не читает `extlinux.conf` и не ищет ядро в файловой системе: это выполняется на более позднем этапе U-Boot.

Корневая файловая система

Определение

Rootfs (root filesystem) — корневая файловая система, содержащая пользовательское пространство Linux: программы, библиотеки, конфигурацию и каталоги, необходимые после запуска ядра.

После инициализации ядро монтирует rootfs и запускает первый процесс, обычно систему инициализации. В rootfs находятся:

- исполняемые программы в `/bin`, `/sbin` и `/usr/bin`;
- библиотеки в `/lib` и `/usr/lib`;
- конфигурационные файлы в `/etc`;
- файлы устройств в `/dev`;
- временные данные и точки монтирования;
- домашние каталоги пользователей в `/home`.

Без rootfs ядро не сможет запустить пользовательские приложения и системные службы. Подходящий архив можно выбрать на странице регулярных сборок ALT Linux для riscv64. Для этой работы предлагается сборка с Xfce, если преподаватель не указал другой вариант.

Конфигурация Extlinux

Определение

extlinux.conf — текстовый файл конфигурации, который сообщает U-Boot пути к ядру и DTB, а также параметры командной строки ядра.

Простейшая конфигурация для подготовленных компонентов выглядит так. Метка R00TFS соответствует разметке из лабораторной №6 и не зависит от номера устройства, присвоенного ядром:

```
default linux-lab5
timeout 50
```

```
label linux-lab5
    menu label ALT Linux (метка-автора)
        kernel /Image
        fdt /sun20i-d1-lichee-rv-dock.dtb
        append root=LABEL=R00TFS rootwait console=ttyS0,115200
```

Параметры имеют следующее назначение:

- `kernel /Image` задаёт путь к ядру на загрузочном разделе;
- `fdt /sun20i-d1-lichee-rv-dock.dtb` задаёт путь к DTB;
- `root=LABEL=R00TFS` указывает раздел с корневой файловой системой по устойчивой метке;
- `rootwait` предписывает ядру дожидаться появления корневого устройства;
- `console=ttyS0,115200` включает вывод консоли ядра через UART со скоростью 115200 бод.

Внимание

Значения `kernel`, `fdt` и `root` должны соответствовать фактическим путям и разметке будущего носителя. Ошибка в любом из них остановит загрузку системы.

Порядок выполнения

Подготовка рабочего каталога и инструментов

Создайте отдельный каталог работы и проверьте инструменты. Не выполняйте сборку в единственном сохранённом каталоге из лабораторной №4:

```
mkdir -p ~/lab5-work
cd ~/lab5-work
command -v git make patch gcc bison flex swig dtc file sha256sum xz \
    riscv64-linux-gnu-gcc python3 rpm2cpio cpio rsync
python3 -c 'import setuptools, pkg_resources'
swig -version
riscv64-linux-gnu-gcc --version
```

Если зависимости отсутствуют, установите их из уже настроенных репозиторий или используйте подготовленное преподавателем окружение:

```
apt-get install git make patch gcc bison flex gcc-riscv64-linux-gnu swig dtc xz \
    python3-module-setuptools python3-module-pkg-resources cpio rsync \
    libssl-devel
```

Внимание

Если command `-v python3` указывает на пользовательскую установку, она может скрывать системные модули. Сначала добейтесь успешного выполнения проверки импорта. Не смешивайте пакеты разных версий Python в одной сборке.

Получение DTB

Возьмите DTB Lichee RV Dock из результатов лабораторной №4 и убедитесь, что его формат и поле `model` определены верно:

```
file arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
dtc -I dtb -O dts \
    arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb \
    > lichee-rv-dock.dts
grep -F 'model = "Sipeed Lichee RV Dock";' lichee-rv-dock.dts
```

Проверка

Ожидаемый результат. Утилита `file` должна определить файл как Device Tree Blob, а не как пустой файл или обычный текст.

Сборка OpenSBI

На сайте сообщества `linux-sunxi` приведён вариант сборки компонентов для Lichee RV. Получите исходный код OpenSBI, перейдите на проверенную ревизию и соберите динамическую прошивку:

```
git clone https://github.com/riscv-software-src/opensbi
cd opensbi
git checkout 3593a5facc4c6938b90429a6973ba9ee21fc5899
git rev-parse HEAD
/usr/bin/time -v -o ../opensbi-build.time \
    make CROSS_COMPILE=riscv64-linux-gnu- PLATFORM=generic FW_PIC=y \
    > ../opensbi-build.log 2>&1
cd ..
```

Результат сборки, используемый U-Boot, находится по пути:

```
opensbi/build/platform/generic/firmware/fw_dynamic.bin
```

Проверьте результат:

```
file opensbi/build/platform/generic/firmware/fw_dynamic.elf
stat -c '%n %s bytes' opensbi/build/platform/generic/firmware/fw_dynamic.bin
sha256sum opensbi/build/platform/generic/firmware/fw_dynamic.bin
```

Сборка U-Boot

Получите ветку U-Boot для Allwinner D1 и перейдите на проверенную ревизию:

```
git clone https://github.com/smaeul/u-boot -b dl-wip
cd u-boot
git checkout 2e89b706f5c956a70c989cd31665f1429e9a0b48
git rev-parse HEAD
make CROSS_COMPILE=riscv64-linux-gnu- lichee_rv_dock_defconfig
```

Ветка dl-wip создана до изменения API в SWIG 4.4. Если установлен SWIG 4.4, примените проверенную совместимую поправку к трём вызовам генератора:

```
patch -p1 <<'PATCH'
diff --git a/scripts/dtc/pylibfdt/libfdt.i_shipped
↪ b/scripts/dtc/pylibfdt/libfdt.i_shipped
--- a/scripts/dtc/pylibfdt/libfdt.i_shipped
+++ b/scripts/dtc/pylibfdt/libfdt.i_shipped
@@ -1040,1040 @@
-         resultobj = SWIG_Python_AppendOutput(resultobj, buff);
+         resultobj = SWIG_Python_AppendOutput(resultobj, buff, 0);
@@ -1079,1079 @@
-         resultobj = SWIG_Python_AppendOutput(resultobj, val);
+         resultobj = SWIG_Python_AppendOutput(resultobj, val, 0);
@@ -1095,1095 @@
-         resultobj = SWIG_Python_AppendOutput(resultobj, val);
+         resultobj = SWIG_Python_AppendOutput(resultobj, val, 0);
PATCH
```

Зафиксируйте время commit как воспроизводимую временную метку, затем запустите сборку с OpenSBI:

```
export SOURCE_DATE_EPOCH="$(git show -s --format=%ct HEAD)"
JOBS=4
/usr/bin/time -v -o ../u-boot-build.time \
    make CROSS_COMPILE=riscv64-linux-gnu- \
    OPENSBI=../opensbi/build/platform/generic/firmware/fw_dynamic.bin \
    -j"$JOBS" > ../u-boot-build.log 2>&1
file u-boot-sunxi-with-spl.bin u-boot u-boot.dtb
stat -c '%n %s bytes' u-boot-sunxi-with-spl.bin
sha256sum u-boot-sunxi-with-spl.bin
cd ..
```

После успешной сборки в каталоге u-boot появится комбинированный образ u-boot-sunxi-with-spl.bin.

Проверка

Проверка сборки. file должен определить комбинированный образ как Allwinner eGON.BT0 Boot Image (RISC-V), а файл u-boot — как RISC-V ELF64. Одного ненулевого размера недостаточно.

Внимание

Ошибка SWIG_Python_AppendOutput означает несовместимость старого интерфейсного файла со SWIG 4.4. Ошибки No module named pkg_resources или No module named setuptools относятся к выбранному Python, а не к кросс-компилятору. Не подменяйте эти ошибки случайной установкой модулей в разные версии Python.

Получение готового U-Boot из репозитория

Вместо изменения системных репозиториев получите подготовленный преподавателем RPM либо проверенный пакет непосредственно из публичного rsync-архива. Следующий вариант был проверен для пакета 2024.01-alt8.git8c36243b6c:

```
rsync -avL \  
  rsync://rsync.altlinux.org/ALTLinux/ports/riscv64/Sisyphus/noarch/RPMS.clas_\  
  ↪ sic/u-boot-sunxi-riscv-2024.01-alt8.git8c36243b6c.noarch.rpm\  
  ↪ \  
  ./\  
mkdir -p u-boot-package\  
cd u-boot-package\  
rpm2cpio ../u-boot-sunxi-riscv-2024.01-alt8.git8c36243b6c.noarch.rpm \  
  | cpio -idm\  
cd ..
```

Проверьте метаданные RPM, найдите вариант для Lichee RV Dock и сравните его размер, версию и контрольную сумму с самостоятельно собранным образом:

```
rpm -qpi u-boot-sunxi-riscv-2024.01-alt8.git8c36243b6c.noarch.rpm\  
PACKAGE_UBOOT=u-boot-package/usr/share/u-boot/lichee_rv_dock/u-boot-sunxi-with-_  
  ↪ spl.bin\  
file u-boot/u-boot-sunxi-with-spl.bin "$PACKAGE_UBOOT"  
stat -c '%n %s bytes' u-boot/u-boot-sunxi-with-spl.bin "$PACKAGE_UBOOT"  
sha256sum u-boot/u-boot-sunxi-with-spl.bin "$PACKAGE_UBOOT"  
strings u-boot/u-boot-sunxi-with-spl.bin | grep -m1 'U-Boot 20'  
strings "$PACKAGE_UBOOT" | grep -m1 'U-Boot 20'
```

Совпадение контрольных сумм не требуется: версии исходного кода и параметры сборки могут различаться.

Подготовка extlinux.conf

Создайте макет загрузочного раздела и конфигурацию на основе приведённого примера. Укажите согласованную с преподавателем метку, по которой можно определить автора будущего образа:

```
mkdir -p prepared/B00T/extlinux
cp /путь/до/Image prepared/B00T/Image
cp /путь/до/sun20i-d1-lichee-rv-dock.dtb prepared/B00T/
```

Сохраните следующий шаблон как prepared/B00T/extlinux/extlinux.conf. Замените текст ФИО-группа согласованной меткой автора. При необходимости шаблон можно дополнить другими параметрами ядра или загрузочными записями:

```
default linux-lab5
timeout 50

label linux-lab5
  menu label ALT Linux (ФИО-группа)
  kernel /Image
  fdt /sun20i-d1-lichee-rv-dock.dtb
  append root=LABEL=R00TFS rootwait console=ttyS0,115200
```

Просмотрите готовый файл и убедитесь, что указанные в нём ядро и DTB существуют в макете B00T:

```
cat prepared/B00T/extlinux/extlinux.conf
ls -l prepared/B00T/Image \
    prepared/B00T/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Проверка конфигурации. В файле должны присутствовать директивы label, kernel, fdt и append; значения путей должны совпадать с именами подготовленных файлов и будущей разметкой носителя.

Получение rootfs

Скачайте указанный преподавателем архив rootfs или предложенную регулярную сборку ALT Linux для riscv64. Имя latest меняет содержимое со временем, поэтому сразу зафиксируйте дату получения, размер, SHA-256 и версию внутри архива. Не извлекайте rootfs на этом этапе:

```
R00TFS=regular-xfce-latest-riscv64.tar.xz
file "$R00TFS"
stat -c '%n %s bytes' "$R00TFS"
sha256sum "$R00TFS"
xz -t "$R00TFS"
```



```
tar -tJf "$R00TFS" > rootfs-files.txt
tar -x0Jf "$R00TFS" ./etc/os-release
grep -E '^\./usr/(sbin/init|lib/systemd/systemd)$' rootfs-files.txt
tar -tvJf "$R00TFS" ./usr/sbin/init ./usr/lib/systemd/systemd
```

Успешный `xz -t` подтверждает целостность потока XZ, а `tar -tJf` — читаемость структуры архива. Для проверенной сборки `/usr/sbin/init` является символической ссылкой на `../lib/systemd/systemd`; при распаковке в следующей лабораторной необходимо сохранить ссылки, владельцев и права.

Итоговая предполётная проверка

Соберите подготовленные файлы в одном рабочем каталоге и создайте контрольный манифест. Для большого `rootfs` допустимо оставить исходный файл на месте, но его точный путь и SHA-256 должны быть записаны в отчёте:

```
mkdir -p prepared/loaders
cp opensbi/build/platform/generic/firmware/fw_dynamic.bin prepared/loaders/
cp u-boot/u-boot-sunxi-with-spl.bin prepared/loaders/u-boot-built.bin
cp "$PACKAGE_UBOOT" prepared/loaders/u-boot-alt-package.bin
cd prepared
sha256sum B00T/Image B00T/sun20i-d1-lichee-rv-dock.dtb \
    B00T/extlinux/extlinux.conf loaders/fw_dynamic.bin \
    loaders/u-boot-built.bin loaders/u-boot-alt-package.bin \
    > SHA256SUMS
sha256sum -c SHA256SUMS
cd ..
```

Внимание

Сохраните ядро, DTB, загрузчик, `extlinux.conf` и архив `rootfs`. Все эти компоненты потребуются при создании образа в следующей лабораторной работе.

Задание

1. Получите DTB Lichee RV Dock из результатов сборки ядра и проверьте тип файла.
2. Соберите OpenSBI и U-Boot для Lichee RV Dock.
3. Получите аналогичный готовый загрузчик из репозитория Sisyphus-riscv64.
4. Сравните размеры и контрольные суммы самостоятельно собранного и пакетного загрузчиков.
5. Скачайте с указанного преподавателем ресурса архив `rootfs`.
6. Подготовьте `extlinux.conf`, указав согласованную метку автора будущего образа.

7. Сохраните ядро, DTB, оба варианта загрузчика, `extlinux.conf` и архив `rootfs`.
8. Создайте манифест с источником, версией или `commit`, размером и SHA-256 каждого компонента.
9. Выполните предполётную проверку макета `BOOT` и продемонстрируйте преподавателю подготовленные компоненты.

Дополнительные задания

Вариант 1. Паспорт комплекта загрузки

Для Image, DTB, `extlinux.conf`, OpenSBI, самостоятельно собранного и пакетного U-Boot и `rootfs` составьте таблицу «назначение — источник — версия или `commit` — размер — SHA-256 — формат». Создайте `SHA256SUMS`, проверьте его через `sha256sum -с`, затем в отдельной копии измените один текстовый файл и подтвердите обнаружение несовпадения. Исходный комплект после опыта должен снова проходить проверку.

Вариант 2. Аудит DTB платы

Декомпилируйте DTB и найдите `model`, `compatible`, CPU, выбранный UART, контроллеры SD/MMC и хотя бы один узел со статусом `disabled`. Объясните отсутствие статического узла памяти, если размер DRAM добавляется загрузчиком. Для намеренной ошибки проверьте DTB близкой платы, например MangoPi MQ Pro: проверка точного значения `model = "Sipeed Lichee RV Dock"` должна завершиться ненулевым кодом.

Вариант 3. Дополнение шаблона `extlinux.conf`

Создайте вторую загрузочную запись на основе готового шаблона: задайте другую строку `label` и `menu label`, но используйте те же проверенные Image, DTB и `root=LABEL=R00TFS`. Объясните назначение `default` и `timeout`, а также как назначить одну из записей загрузочной записью по умолчанию. После проверки удалите дополнительную запись или сохраните исходный шаблон отдельно.

Вариант 4. Самостоятельный и пакетный U-Boot

Сравните формат, полный размер, SHA-256 и строку версии двух образов. Объясните, почему одинаковый заголовок `eGON` и размер `SPL 73728` байт не требуют совпадения полных образов. Типовая ошибка — объявить один файл повреждённым только из-за разных хешей; исправление — сначала сопоставить версии, `commit`, параметры и источник OpenSBI.

Вариант 5. Аудит архива `rootfs`

Без распаковки проверьте `rootfs` командами `xz -t`, `tar -tJf` и `tar -tvJf`, найдите `os-release`, `init`, системный менеджер и специальные файлы `/dev`. Создайте усечённую копию небольшого размера командой

`head -c 16M "$R00TFS" > rootfs.truncated.tar.xz`, подтвердите ошибку `Unexpected end of input`, затем удалите только усечённую копию. Объясните, почему `file` сам по себе не доказывает целостность архива.

Вариант 6. Воспроизводимая сборка U-Boot

Выполните две чистые сборки одной ревизии U-Boot с одинаковыми OpenSBI, тулчейном, `defconfig`, числом заданий и `SOURCE_DATE_EPOCH`, взятым из `commit`. После каждой сборки сохраните SHA-256 `u-boot-sunxi-with-spl.bin` и сравните файлы через `cmp -s`. Затем объясните, почему без фиксированной эпохи в строку версии попадает текущее время и хеш может измениться. Все сравнения выполняйте на копии исходников; сохранённый результат обязательной части не удаляйте.

Контрольные вопросы

1. Что такое Device Tree и какую роль оно выполняет при загрузке Linux?
2. Чем файл `.dts` отличается от `.dtb`?
3. Для чего нужны SPL и U-Boot proper в цепочке загрузки?
4. Какую роль выполняет OpenSBI при загрузке RISC-V?
5. Какая информация содержится в файле `extlinux.conf`?
6. Почему путь `root=LABEL=R00TFS` устойчивее имени вида `/dev/mmcblk0p2`?
7. Почему `file` и ненулевой размер не заменяют SHA-256 и проверку содержимого?
8. Для чего при воспроизводимой сборке используется `SOURCE_DATE_EPOCH`?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему цепочки загрузки Lichee RV Dock;
- путь и результат проверки DTB;
- команды и результаты сборки OpenSBI и U-Boot;
- сведения о пакетном U-Boot и сравнение двух вариантов;
- содержимое подготовленного `extlinux.conf`;
- имя, размер и контрольную сумму архива `rootfs`;
- версии или `commit` исходников и версии основных инструментов;
- результат проверки `model` и `compatible` в DTB;
- манифест компонентов и результат `sha256sum -s`;
- перечень сохранённых компонентов;
- результаты выбранного дополнительного задания, если оно выполнялось;
- ответы на контрольные вопросы;

- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.